



School of Future Tech

Case Study Report

on

Bike Docking Station Asset Balancer

by

Name: Himanshu katiyara

Roll no: 150096725195

About me: B.Tech 2025-2029 Demmis Hassabis Semester Two
Project:

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

In rapidly expanding modern metropolitan cities such as Pune and Bangalore, traditional public transportation infrastructure frequently struggles with the "last-mile connectivity" problem. Micromobility platforms—specifically dockless and docked bicycle-sharing networks—have emerged as a sustainable, eco-friendly, and cost-effective remedy.

Thousands of commuters rely daily on these networks to unlock a bicycle via a smartphone application at one urban intersection, navigate through traffic, and secure it at another hub close to their destination.

However, managing such a decentralized fleet introduces massive operational hurdles behind the scenes:

- Asymmetrical commuter traffic causes certain commercial hubs to become completely choked with overflowing bicycles while residential nodes run empty by mid-morning.

- Fleet assets suffer from continuous wear-and-tear, requiring real-time damage reporting and structured repair logistics.

- Field operations require complex asset rebalancing, safe commuter route mapping, and nighttime truck-loading optimizations to keep the system sustainable.

This case study reviews the design and deployment of the **Bike Docking Station Asset Balancer**, an enterprise-level simulation system built in C++ utilizing high-performance Data Structures and Algorithms (DSA) to resolve these operational bottlenecks.

2. Problem Statement / Case Background (Abstract)

The core objective of the **Bike Docking Station Asset Balancer** is to manage a complex municipal bicycle-sharing ecosystem efficiently. The system must process thousands of concurrent assets while ensuring low-latency data access and mathematically optimized field distribution.

The application must address four critical areas of operational friction:

1. **Asset Visibility:** Maintaining an instantaneous registry of bike locations, health indices, and locking states.
2. **Fault Tolerance:** Providing instant restoration mechanics for accidental checkouts or hardware malfunctions (such as jammed physical locks).
3. **Logistical Rebalancing:** Identifying depleted docking infrastructure, routing transport trucks along safety-optimized urban paths, and packing vehicle payloads mathematically to eliminate space and fuel waste.

By abstracting these real-world conditions into discrete computer science problems, this system guarantees that no commuter encounters an empty rack or a non-functional bicycle during peak hours.

3. Problem Statement / Case Study Design

The application is architected around a unified **System Controller Pattern** implemented via a centralized manager class in C++. This modular framework separates structural entities from algorithm engines to preserve memory boundaries and maximize execution speed.

Core Entity Definitions

Bike Structure: Encapsulates unique alphanumeric identifiers, string-based geolocation tracking (current node/station assignment), and strict state machines (`locked`, `unlocked`, `in-use`, `broken`).

Station Structure: Manages the capacity metrics of physical infrastructure. It utilizes inline evaluation mechanics to compute real-time operational demand based on vacant docking slots.

Edge Structure: Models the structural link between urban intersections, mapping physical proximity alongside dynamic safety hazard indicators.

4. Methods & Algorithms Technology Applied

The core strength of the system rests upon mapping real-world physical operations to appropriate abstract data structures. This selection guarantees optimal time and space complexity boundaries.

Technical Mapping & Complexity Matrix

Algorithmic Breakdown

Dijkstra’s Pathfinding Strategy: The urban landscape is modeled as a weighted graph $G=(V,E)$. To ensure commuter safety, the priority queue (Min-Heap) extracts nodes based on accumulated road hazard values rather than physical distance. Roads with heavily congested traffic carry high safety weights, forcing the algorithm to reroute the user through low-risk, dedicated bicycle paths.

0/1 Knapsack Optimization Strategy: Payload rebalancing is solved via a bottom-up Dynamic Programming table. Given a truck weight ceiling W and arrays representing bicycle rebalancing values and space constraints, the algorithm fills a 2D matrix ($dp[i][w]$). It evaluates whether to load or bypass an asset to maximize the utility of every night-shift truck run.

5. Problem Statement / Case Study
Implementation Details and Snapshots

Implementation Methodology

The system was written in standard-compliant C++ using the Standard Template Library (STL) to minimize low-level memory handling overhead and prevent memory leaks. Execution logic is funneled through an interactive Command Line Interface (CLI) loop.

To demonstrate functional accuracy during testing, the initialization layer populates the system memory with real-world infrastructure parameters:

Automated registration of items (Bikes 101, 102, 103).

Diverse urban destination metrics (Stations with varying availability levels).

A multi-edge routing topology that explicitly matches safety risks to alternative paths.

System Verification Snapshots (Conceptual Output Log)

When the executable runs, the console presents an operational menu interface that validates the underlying logic during user testing:

Plaintext

=====

BIKE DOCKING SYSTEM MENU

=====

1. Track a Bike
2. Checkout a Bike
3. Undo Last Checkout
4. Report a Broken Bike
5. Fix Next Broken Bike
6. View Emptiest Stations
7. Find Safest Route (Test 1 -> 2)
8. Run Truck Loader Optimization
0. Exit System

Enter your choice:

Note on Functional Verification:

Selecting **Option 6 (Station Sorter)** confirms that the system organizes locations by vacancy levels, moving regions with high numbers of open docking slots to the top of the sorting list.

Selecting **Option 7 (Safest Route)** proves the system's pathfinding logic. It successfully rejects a direct road connecting points 1 and 2 due to its high danger level (weight 5.0), choosing instead to route through point 3 via two consecutive low-risk bicycle lanes (weight 1.0 + 1.0 = 2.0).

6. Problem Statement / Case Study Results and Conclusion

The successful compilation and execution of the simulation demonstrate that combining structured data architectures with classical optimization algorithms produces a highly effective logistics platform.

Key Findings

1. **Instant Asset Control:** Utilizing Hash Maps for tracking eliminates calculation latency. Lookups run in constant time, meaning data retrieval stays fast even as the bike fleet expands from dozens to thousands of units.
2. **Deterministic Payload Optimization:** Implementing a Dynamic Programming framework for truck loading guarantees that support vehicles operate at maximum capacity. This minimizes fuel consumption and shortens the operational time needed for nightly asset rebalancing.
3. **Safety-Centric Routing:** Modifying Dijkstra's algorithm to evaluate safety constraints rather than just physical distance shows that urban transit networks can prioritize human safety within complex logistics environments.

Conclusion

This project successfully simulates an automated bike-sharing network. The design choice proves that complex municipal operational challenges can be broken down into discrete data structures and solved efficiently. The system serves as a solid foundation for real-world micromobility management applications.

7. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.

3. Shaheen, S., Cohen, A., & Martin, E. (2013). Bikesharing in North America: Institutional Models, Impacts, and Lessons Learned. *Transportation Research Record*, 2387(1), 103-112.
4. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.